Cryptography, Theory and Practice CIS 4930/6371 (Fall 2026)

Instructor: **Dr. Attila A. Yavuz**
TA: ***Sean Hernandez***

October 9, 2026

Homework 4 (Due: *Oct.* 16, 2026 – 23 : 59)[100 Points]

(Upload your solutions to the CANVAS)
Requirements: HW groups are 2-3 people. Each student should contribute all parts of the answers. If one group member drops, the others are equally responsible (each HW can be handled by a single student if needed, so no excuse for missing a partner (if you are doing in group of 2), see syllabus for breaking-up with pair programming partner). Directly borrowing (e.g., copy-paste) from any material and putting in solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations. All your solutions will be passed through MOSS and Turnitin for plagiarism check.**
Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also make research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially be not sufficient.
Remark: HW should be prepared by a text editor (e.g., Microsoft Word or Latex). Handwritten submissions are not accepted. In your answer key, please indicate the point of each question as it is done in this HW document. Preferably, take questions from here, plug them into your answer key with their sub scores, and provide your answer below them. This will facilitate the grading.
As indicated in Syllabus, **no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.**

0.1 Programing Assignment – ElGamal Encryption & ECDSA via OpenSSL. [100 points]

This assignment will have you implement an authenticated encryption scheme, using ElGamal for Encryption and ECDSA for signing. Recall previous lectures on two-party key exchange (such as Diffie-Hellman and ECDH), digital signature schemes, and Elliptic curve cryptography. This assignment will put those concepts into practice using ElGamal and ECDSA, where you will implement ElGamal Key generation, encryption/decryption, and ECDSA signing/verification using OpenSSL.
Please fill in minimal implementation and configuration details by yourself independently (and this is how learning happens).

Implementation Setting

- Use the Elliptic Curve with the following specification on OpenSSL: *NID_secp192k1*.

- All the EC_GROUP and EC_POINT data are based on the above specification.
- To ensure the accuracy of your code, utilize unsigned char, BN, and EC Point for operations, reserving Hex exclusively for file output. So, perform the conversions accordingly.
- **You have access to all the functions provided in class exercises or homework assignments. Please modify them as necessary. Additionally, for your convenience, we have provided a template that contains all the necessary functions and their description so that you could use them to create your solution ("RequiredFunctionsHW4.c").**
- **Also, feel free to use any function of your choice as long as it produces the correct output. Refer to the test vectors to understand the grading criteria.**

El-Gamal Key Generation: Certificate Authority [10 pt]

(y, Y) ← ElGamal.KeyGen(seed)

1) $y = H(seed)$
2) $Y = y.G$

Certificate Authority (CA) performs the following steps to generate secrete and public key sets:

1. CA reads the seed from a file named "Seed.txt" which contains 32 Bytes of random characters. (Read the message as unsigned char so that you could use the functions provided in the template file)
2. Using a SHA256 Hash function from OpenSSL, it creates the secret key (y).
3. Subsequently, CA generates the PK by performing scalar multiplication with the EC Group generator (G).
   - For the scalar multiplication of "y" and "G", use the provided functions:
   - First create a group based on *NID_secp192k1*.
   - Get the group generator "G" as an *EC_POINT* from the created *EC_GROUP*.
   - The private key must be of the format *BIGNUM*. So, first, convert "y" into Hex and then, into a *BIGNUM* data structure.
   - Perform the multiplication.
4. CA writes the Hex format of SK and PK in files named $\textbf{"SK}_h ex.txt"\ and \textbf{PK}_h ex.txt, respectively.$

Terminal
Compile the key generation part with the following command:
gcc KeyGen.c -lssl -lcrypto -o CertificateAuthorithy
Run your code for the first test files with the following commands:
./CertificateAuthorithy Seed1.txt

ElGamal Encryption: Alice [25 pt]

(C, D) ← ElGamal.Enc(PK=Y, m)

1) $k \leftarrow Random(1^{128})$
2) $C = k.G$
3) $C' = k.Y$

4) $m \rightarrow P_m$
5) $D = C' + P_m$
5) $ciphertext = (C, D)$

Alice performs the encryption on the message as follows:

1. k is a random number selected with 128-bit security strength. In this HW, Alice reads the random k from a file named "**Random_k_Hex.txt**".
   - Technically, k is the Hex representation of a real random number.
   - Read the k as char, so that you can easily convert it to an EC_POINT for encryption.
2. Alice reads the message from a file named **Message.txt**.
   - Technically, P_m is the Hex representation of a real random message on an elliptic curve.
   - Read the m as char, so that you can easily convert it to an EC_POINT for encryption ($m \rightarrow P_m$).
3. Alice reads the PK (**Y**) from the file named "**PK_Hex.txt**".
   - Read Y as char, so that you can easily convert it to an EC_POINT for encryption.
4. Alice performs $C = k.G$
   - where k is of BIGNUM format and G is the generator of the EC Group.
5. Alice performs $C' = k.Y$
   - where k is of BIGNUM format and Y is of EC_POINT format.
6. To obtain P_m, Alice only needs to convert the message (read from file **Message.txt**) from Hex to an EC_POINT.
7. Alice performs $D = C' + P_m$
   - where the above addition is adding two EC_POINTs. Use the provided functions.
8. After obtaining the ciphertext (C, D), Alice writes the Hex format of C and D in files named **"C_Hex.txt"** and **"D_Hex.txt"**, respectively.
   - Using the provided functions, convert the "C" and "D" parts of the ciphertext into Hex format and then write it in the files.

ECDSA Signing: Alice [20 pt]

Alice performs a signature on the ciphertext as follows:

You are given Alice's ECDSA secret key for signing as **"alice_sk.txt"**.

1. Alice reads her secret key from a file named **"alice_sk.txt"**
   - The secret key file contains the hex value of the key
   - Using the provided functions, you must convert the key to BIGNUM or EC_POINT data structures.
   - After type conversion, you can use the provided functions to set the private/public key in the data type EC_KEY so that you can utilize the keys for signing and verifying procedures.

2. Concatenate the ciphertext C and D as $(C||D)$
    - Use the ciphertext C and D that you produced in the ElGamal Encryption section
    - You can hash the hex representation of the ciphertext
3. Use SHA256 to hash the concatenated ciphertext as $H(C||D)$
    - Use OpenSSL's SHA256 function for hashing
4. Compute the signature of the hash and write to file called **"alice_signature.txt"**
    - Note that Alice signs with her ECDSA secret key.
    - Convert the signature to a Hex string and write it in a file named "signature_alice.txt".

Terminal
Compile the Encryption part with the following command:
gcc alice.c -lssl -lcrypto -o Alice
Run your code with the following commands:
./Alice PK_Hex.txt alice_sk.txt Random_k_Hex.txt Message.txt
In total, Alice should produce 3 files: **C_Hex.txt, D_Hex.txt,** and **alice_signature.txt**

ECDSA Verification [20 pt]

You are given Alice's ECDSA private key as **"bob_pk_hex.txt"**
Using this public key, Bob will read in the signature and validate it before decryption

- Bob reads **"alice_signature.txt"** from file as hex
- Bob reads the ciphertext from file as **"C_hex.txt"** and **"D_hex.txt"**
- Bob concatenates the two ciphertext files as $(C||D)$
- Bob hashes the concatenated ciphertext as $H(C||D)$
- Using Alices ECDSA public key, Bob validates the signature using `ECDSA_verify` functions from OpenSSL.

ElGamal Decryption: Bob [25 pt]

$P_m \leftarrow$ ElGamal.Decrypt(SK=y, ciphertext=(C,D))

1) $C' = y.C$
2) $P_m = D - C'$

Bob performs the Decryption of the ciphertext as follows:

1. Instead of receiving the ciphertext from Alice via ZMQ, we are reading from the files.
2. Bob reads the ciphertext from files **"C_Hex.txt"** and **"D_Hex.txt"**.
    - Read (C, D) as char, so that you can easily convert them to EC_POINTs for decryption.
3. Bob reads his secret key from "**SK_Hex.txt**" file.
    - Read SK as char, so that you can easily convert it from Hex to BIGNUM for decryption.

4. Bob performs $C' = y.C$
    - where y is of BIGNUM format and C is of EC_POINT format.
5. Bob performs $P_m = D - C'$
    - Use the provided functions to invert C' to obtain $-C'$.
    - Use the addition function to add two EC_POINTs $(D, -C')$
6. The final point would be P_m, which is the point version of the message. Convert the obtained P_m to Hex format and write it in a file named **"Plaintext.txt"**.
    - The **Plaintext.txt** content must be the same as the input **"Message.txt"**.

Terminal
Compile the Decryption part with the following command:
gcc Decrypt.c -lssl -lcrypto -o Bob
Run your code with the following commands:
./Bob SK_Hex.txt C_Hex.txt D_Hex.txt

Programming Assignment - Specific Notes:

- An invalid signature must cause Bob to exit the program. If Bob continues to decryption and recovers the Plaintext without verifying the signature (or verifying an invalid signature), you will lose credit.
- Use the Elliptic Curve with specification *NID_secp192k1* on OpenSSL.
- All the EC_GROUP and EC_POINT data are based on the above specification.
- To ensure accuracy, utilize unsigned char, BN, and EC Point for operations, reserving Hex exclusively for file output.
- Files provided to you in this assignment **(Random_k_Hex.txt, alice_sk.txt, alice_pk.txt)** are given as Hex. Use the functions provided in the required functions file **RequiredFunctionsHW4.c** to convert them to the necessary format.
- For your convenience, we have provided template files with necessary functions and descriptions: **RequiredFunctionsHW4.c**
- In the template files, you are free to modify or write your own functions, just ensure you provided the updated template file along with your code submission.
- You can refer to the OpenSSL library documentation `https://www.openssl.org/docs/manmaster/man3/` to learn how to use the functions.
- We have provided a script (VerifyingYourSolutionECDSA.sh) and test vectors that you can use to test the correctness of your final code. Please, only submit your code if it is functional
- In order to use the script, just put your codes, the rest of the provided files along with the "VerifyingYourSolutionECDSA" in one folder and run the following command in the terminal:
bash VerifyingYourSolutionECDSA.sh
- Ensure the output of your code is correctly named as per the instructions.

- You may look at the testing script to understand how your code is being tested. This script is the same one used for grading on our side, however the test vectors will be different
- **You Must submit 3 files for this assignment:**
  KeyGen.c & alice.c & bob.c

Rubric:

- The **"SK_Hex"** file and the **"PK_Hex"** files **[20 pts]**
- The **"C_Hex.txt"** file and the **"D_Hex.txt"** files **[20 pt]**
- The **"alice_signature.txt"** file **[20 pt]**
- Bob **correctly exiting** upon invalid signature **[20 pt]**
- The **"Plaintext.txt"** file **[20 pt]**

0.2 Extra Credit: On the Compaction Schnorr Signature Scheme: [15 points]

Let $(S_1, R_1), ..., (S_n, R_n)$ be n distinct Schnorr signatures computed on distinct messages $M_1, M_2, ..., M_n$ under the same private key K_s. Is it possible to aggregate these signatures and then broadcast it as one single-compact (constant-size) signature to be verified by all the signers?

If your answer is YES, show mathematically (and algorithmically) how it can do. If your answer is NO, similarly, how mathematically (and algorithmically) why not. Your answer should not be handwaving verbal descriptions, and it must include necessary mathematical rigor to showcase possibility or impossibility result. Note that in the class, we showed in detail how Schnorr signature is computed with a correctness verification. You can recall these equations to deduce your conclusions.

Write your solution to **HW4_Group(number).pdf**

Common Assignment Notes:

- Overall you should be submitting 4 files total:
  - **KeyGen.c** for ElGamal Key generation
  - **alice.c and bob.c** for signature/encryption and verification/decryption
  - **HW4_Group(number).pdf** for Extra Credit
- For your convenience, we have provided template files with necessary functions and descriptions:
  - **RequiredFunctionsHW4.c** for Programming Assignment 1
- You can refer to the OpenSSL library documentation `https://www.openssl.org/docs/manmaster/man3/` to learn how to use the functions.
- We have provided test files and verification scripts for both assignments.
- In grading, different keys will be used and everything must work.
- Code that throws errors or does not compile/run on our side receives zero credit.
- Follow all specific file names mentioned above, otherwise there will be 20% deduction for inconvenience.
- Submit your code only if it is functional and passes the provided test cases.